Multi-Dimensional Array in Java

03 November 2026 20:14

Multidimensional array in Java are used to store data in the form of rows and columns.

What are multi - dimensional arrays in Java?
Multi-dimensional array or arrays of arrays where each element of the main array holds another array. They are commonly used to represent data in table or matrix form. Such as grids, charts or spreadsheets.

A multi-dimensional array can be compared to a table. For example, storing marks of students where each row represents a student and each column represents a subject. This way data is organized in rows and columns using a multi-dimensional array.

Two-dimensional (2D) Array

A two dimensional array (2D) array is a type of multi-dimensional array that stores data in the form of rows and columns. It is used to represent table - like structures such as matrices, grids or tables where data is organized in two dimensions only.

Declaration
You can use one of the following syntaxes to declare a 2-dimensional array.

```
DataType[][] arrayName;

Or

DataType arrayName[][];

Allocating memory:

arrayName = new DataType[rows][columns];
```

We can declare and allocate memory for two dimensional array in a single statement as follows:

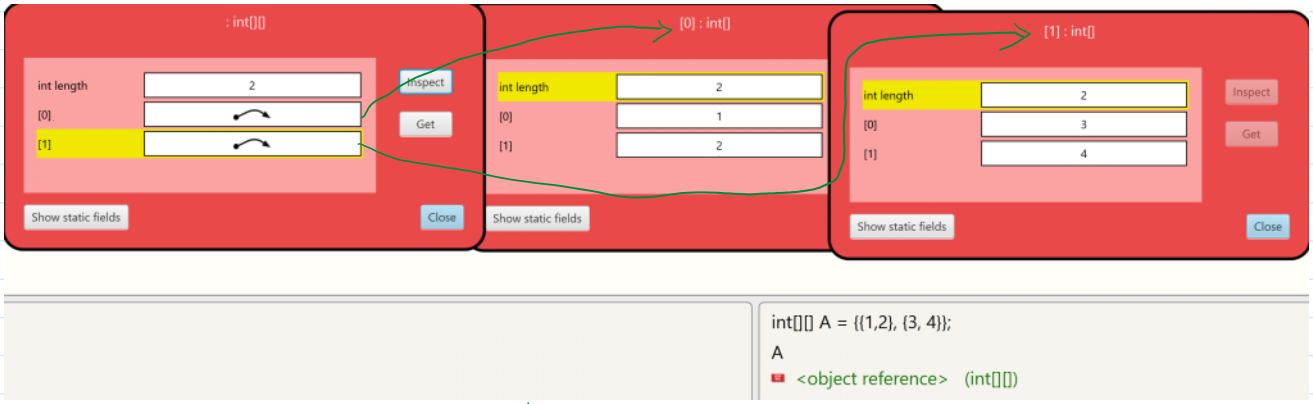
```
Datatype[][] arrayName = new DataType[rows][columns];

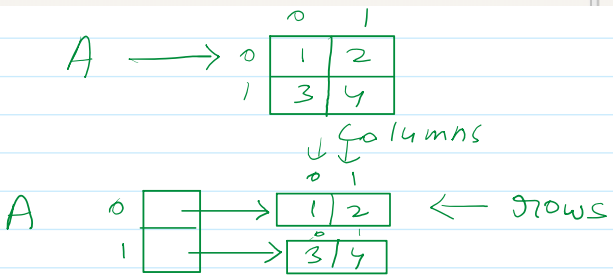
            Or
Datatype arrayName[][] = new DataType[rows][columns];
```

Assigning values (initialization)

```
Datatype arrayName[][] = {{v1, v2, v3…},{…},…};
```

For example:





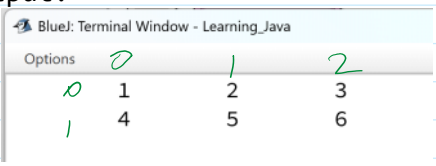

```
package MyArray.MultiDimension;


/**
 * Write a description of class One here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class One
{
    public static void main(String[] args){
        System.out.print("\f");
        int[][] A = {{1,2,3}, {4, 5, 6}};
        int i, j;
        for(i = 0; i < A.length; i++){

            for(j = 0; j < A[i].length; j++){
                System.out.print("\t" + A[i][j]);
            }
            System.out.print("\n");
        }
    }
}
```

number of rows
number of columns
or
number of elements in a row
arrayName.length
arrayName[i].length

Output:




```
package MyArray.MultiDimension;
import java.util.*;
```

```
/**
 * Write a description of class UserInput here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class UserInput
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("\fEnter number of rows: ");
        int rows = sc.nextInt();
        System.out.print("Enter number of columns: ");
        int cols = sc.nextInt();

        int[][] A = new int[rows][cols];
        int r, c;
        for(r = 0; r < A.length; r++){
            for(c = 0; c < A[r].length; c++){
                System.out.print("Enter intgeger value in A["+r+"]["+c+"]:
");
                A[r][c] = sc.nextInt();
            }
        }

        //Display 2-D array
        for(r = 0; r < A.length; r++){
            for(c = 0; c < A[r].length; c++){
                System.out.print("\t" + A[r][c]);
            }
            System.out.print("\n");
        }

    }
}
```

Output:

```
BlueJ: Terminal Window - Learning_Java
Options
Enter number of rows: 3
Enter number of columns: 2
Enter intgeger value in A[0][0]: 10
Enter intgeger value in A[0][1]: 1
Enter intgeger value in A[1][0]: 12
Enter intgeger value in A[1][1]: 14
Enter intgeger value in A[2][0]: 5
Enter intgeger value in A[2][1]: 16
        10      1
        12      14
        5       16
```

WAP to create a 2-D Array by taking number of rows and number of columns, and print the 2-D array. Find the sum of each row and each column.

1	2	3	= 6

and print the 2-D array. Find the sum of each row and each column.

1	2	3	= 6
4	5	6	= 15
5	7	9	

Jagged Array in Java

What is jagged array in Java?
A jagged array is a collection of arrays, where each array may contain a varying number of elements. A two-dimensional array in contrast requires all rows and columns to have the defined length.

Jagged arrays are also known as "ragged arrays" or "irregular arrays". They can be created by specifying the size of each array in the declaration.

For example, a jagged array with three rows can have the first row with three elements, second with two elements and the third with 5 elements.

Jagged Array Declaration
A jagged array is declared by first specifying the number of rows and then allocating each row separately with different column size.

Syntax:

```
Datatype[][] arrayName = new Datatype[numRows][];

arrayName[0] = new Datatype[numColumns1];
arrayName[1] = new Datatype[numColumns2];
…
arrayName[numRows-1] = new Datatype[numColumnsN];
```

The first line creates an array of arrays with numRows rows. Each row is initialized to NULL by default.

The subsequent lines initialize each row of the jagged array. You can create a new one dimensional array of **numColumns** elements for each row and assign it to the corresponding row in the jack array.

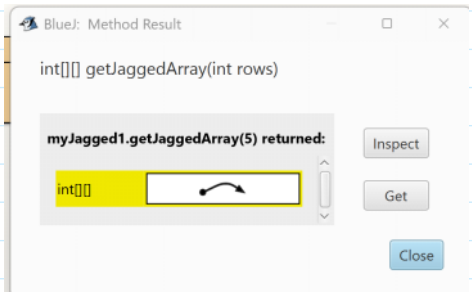
Ways to initialize a Jagged Array
Apart from the standard approach of declaring and initializing a jagged array, there are several different ways to initialize a jagged array in Java.

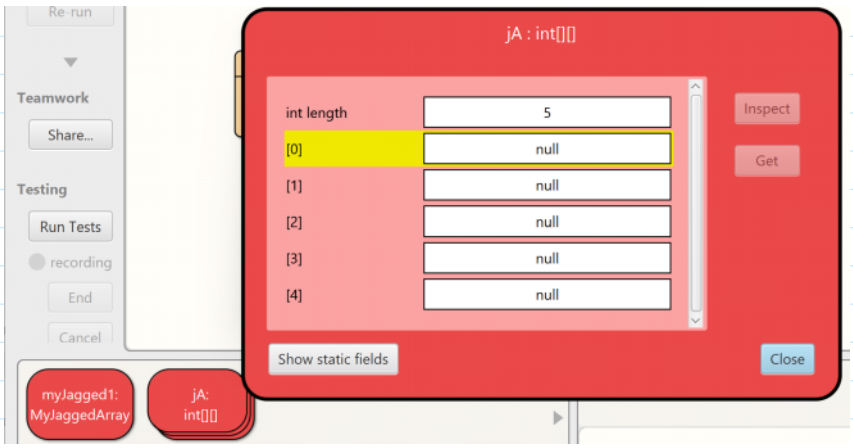
1. Using Array Literals
Literals to initialize the jagged array elements like this directly:

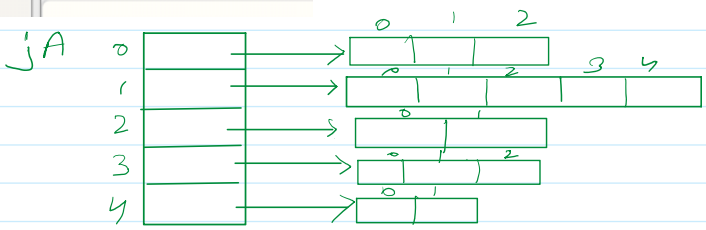
```
package MyArray.MultiDimension;


/**
 * Write a description of class MyJaggedArray here.
```

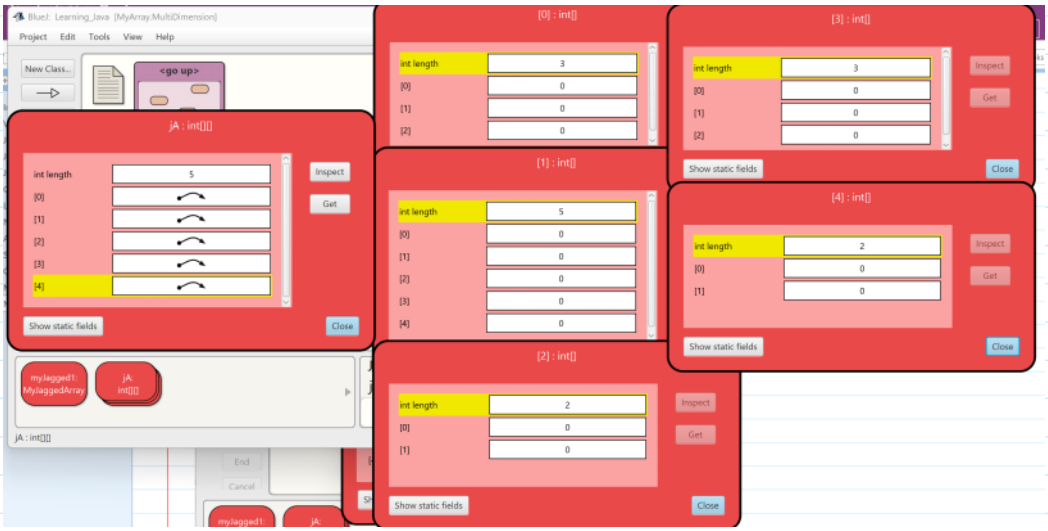
```
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class MyJaggedArray
{
    public int[][] getJaggedArray(int rows){
        int[][] arr = new int[rows][];
        return arr;
    }
}
```








```
jA[0] = new int[3];
jA[1] = new int[5];
jA[3] = new int[3];
jA[2] = new int[2];
jA[4] = new int[2];
```



```
package MyArray.MultiDimension;
import java.util.*;


/**
 * Write a description of class MyJaggedArray here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class MyJaggedArray
{
    static Scanner sc = new Scanner(System.in);
    public int[][] getJaggedArray(int rows){
        int[][] arr = new int[rows][];
        int i, numOfElements;
        for(i = 0; i < rows; i++){
            System.out.print("\nNumber of elements in row " + i+": ");
            numOfElements = sc.nextInt();
            arr[i] = new int[numOfElements];
        }
        return arr;
    }

    public void input(int[][] jA){

        int i, j;
        System.out.print("\nData input in Jagged Array:\n");
        for(i = 0; i < jA.length; i++){
            for(j = 0; j < jA[i].length; j++){
                System.out.print("Enter value in jA["+i+"]["+j+"]: ");
                jA[i][j] = sc.nextInt();
            }
        }
    }

    public void printJaggedArray(int[][] jA){
        int i, j;
        System.out.print("\nData in Jagged Array:\n");
```

```
        for(i = 0; i < jA.length; i++){
            for(j = 0; j < jA[i].length; j++){
                System.out.print("\t"+jA[i][j]);
            }
            System.out.print("\n");
        }
    }
}
```

Output:

```
   Number of elements in row 0: 5

   Number of elements in row 1: 3

   Number of elements in row 2: 2

   Data input in Jagged Array:
   Enter value in jA[0][0]: 10
   Enter value in jA[0][1]: 20
   Enter value in jA[0][2]: 30
   Enter value in jA[0][3]: 40
   Enter value in jA[0][4]: 50
   Enter value in jA[1][0]: 100
   Enter value in jA[1][1]: 200
   Enter value in jA[1][2]: 300
   Enter value in jA[2][0]: 1000
   Enter value in jA[2][1]: 2000

   Data in Jagged Array:
      10  20  30  40  50
      100 200 300
      1000   2000
```

Jagged array using Java 8 Streams

With Java 8 streams you can create and initialize a jar array in a single line using the Arrays.stream and toArray methods like this:

```
package MyArray.MultiDimension;
import java.util.stream.*;
import java.io.*;
import java.util.*;


/**
 * Write a description of class MyJaggedArray here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class MyJaggedArray
{
    static Scanner sc = new Scanner(System.in);
    public int[][] getJaggedArray(int rows){
```

```
        int[][] arr = new int[rows][];
        int i, numOfElements;
        for(i = 0; i < rows; i++){
            System.out.print("\nNumber of elements in row " + i+": ");
            numOfElements = sc.nextInt();
            arr[i] = new int[numOfElements];
        }
        return arr;
    }

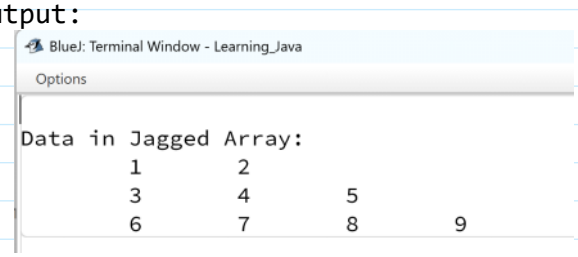
    public void input(int[][] jA){

        int i, j;
        System.out.print("\nData input in Jagged Array:\n");
        for(i = 0; i < jA.length; i++){
            for(j = 0; j < jA[i].length; j++){
                System.out.print("Enter value in jA["+i+"]["+j+"]: ");
                jA[i][j] = sc.nextInt();
            }
        }
    }

    public void printJaggedArray(int[][] jA){
        int i, j;
        System.out.print("\nData in Jagged Array:\n");
        for(i = 0; i < jA.length; i++){
            for(j = 0; j < jA[i].length; j++){
                System.out.print("\t"+jA[i][j]);
            }
            System.out.print("\n");
        }
    }

    public int[][] getJaggedArrayUsingStream(){
        int[][] jA = Stream.of(
            new int[]{1,2},
            new int[]{3,4,5},
            new int[]{6,7,8,9}).toArray(int[][]::new);
        return jA;
    }
}
```

Output:



Here we make three - row jagged array with two elements in the first row, 3 in the second row and four in the third row. The **Arrays.stream** method converts each one-dimensional array into a stream of integers and then the **toArray** method collects the stream into a jagged array.

Algebraic identities :

toArray method collects the stream into a jagged array.

Algebraic identities.

$(a+b)^0 = 1$

coefficients

$(a+b)^1 = 1 \cdot a^1 + 1 \cdot b^1$

$(a+b)^2 = 1a^2b^0 + 2ab + 1a^0b^2$ ← the index of 1st variable is decrementing, and the index of second variable is incrementing

$(a+b)^3 = 1a^3 \cdot b^0 + 3a^2 \cdot b^1 + 3a^1 \cdot b^2 + 1a^0 \cdot b^3$

$(a+b)^4 = 1a^4 \cdot b^0 + 4a^3 \cdot b^1 + 6a^2 \cdot b^2 + 4a^1 \cdot b^3 + 1a^0 \cdot b^4$

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1
:
```

All these values are called co-efficients of terms.

WAP, in Java to ask the user number of rows and use the concept of jagged Array to store coefficient of terms. And print the jagged Array.